

Report: Programming Assignment 1

Kaarmukilan B - 23B1215
Nandana M A - 23B1216
Maintranya Patil - 23B1239
Sreesti Sahoo - 23B1267

September 7, 2026

1 Experimental Setup

Table 1: Experimental platform

Processor	13th Gen Intel(R) Core(TM) i5-13420H
Architecture	Intel x86-64
L1-D cache	40 KiB per core (320 KiB total across 8 cores)
L2 cache	7 MiB total (5 instances)
LLC / L3 cache	12 MiB (1 instance)
Compiler	gcc/g++ (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0
OS	Ubuntu 24.04.4 LTS

2 Baseline Profiling

2.1 L1 Data Cache Size

Command used: `lscpu | grep "L1d cache"`.

Reported L1-D cache: **320 KiB (8 instances)**, implying 40 KB per core.

2.2 Profiling Naive 2D Convolution

The baseline implementation uses triple nested loops (rows, columns, kernel) with no optimizations. Cache misses and instructions were measured with **perf**. The L1-D MPKI for different matrix sizes and kernel sizes are shown in Table 2.

Table 2: Naive L1-D MPKI

Matrix Size	K=3	K=5
512	0.12149	0.41713
1024	0.14354	0.09644
2048	0.14094	0.12942

3 Loop Reordering and Unrolling

3.1 Motivation and Implementation

The motivation was to improve memory access patterns. Reordering loops so that the row index is outer and column index is inner improves spatial locality because consecutive elements in a

row are contiguous in memory. The kernel loops were moved outside. For unrolling, since the matrix width W is always a multiple of 8, the inner column loop was unrolled by a factor of 8, requiring 8 separate accumulators to avoid mixing output values. All output elements were initialized to zero before accumulation.

3.2 Effectiveness Metric

Speedup relative to the naive implementation is used as the primary metric, as it directly reflects the overall performance improvement.

3.3 Results

Table 3 lists the speedups for reordering and unrolling for various matrix and kernel sizes.

Table 3: Speedup for Loop Reordering and Unrolling

Matrix Size	K	Reorder	Unroll
512	3	1.70×	3.78×
512	5	1.58×	3.28×
1024	3	1.50×	3.21×
1024	5	1.95×	3.95×
2048	3	1.66×	3.65×
2048	5	1.96×	3.96×
4096	3	1.60×	3.45×
4096	5	1.36×	2.71×

3.4 Analysis

Reordering improved speedup modestly (1.36–1.96×) by enhancing spatial locality. Unrolling delivered higher speedups (2.71–3.96×) by reducing loop overhead and increasing instruction-level parallelism. Reordering alone was less effective because the L1 cache (40 KB) is small relative to the working set of larger matrices, so the benefit of improved locality was limited.

4 Tiled 2D Convolution

4.1 Implementation

The input matrix is divided into square tiles of size `TILE_SIZE`. Each tile is processed completely before moving to the next, improving temporal locality of the kernel and nearby input elements. Tile sizes tested: 8, 16, 32, 64, 128, 256.

4.2 Cache Behavior (MPKI)

Tables 4 and 5 report L1-D MPKI for $K=3$ and $K=5$ across matrix sizes and tile sizes.

Table 4: L1-D MPKI for K=3 with Tiling

Tile Size	512	1024	2048
8	0.143518	0.178930	0.162925
16	0.158791	1.548491	0.929331
32	0.597696	0.918674	0.505296
64	1.073239	0.346298	0.965489
128	0.171674	0.151357	0.186503
256	0.859067	0.206871	0.445984

Table 5: L1-D MPKI for K=5 with Tiling

Tile Size	512	1024	2048
8	0.453036	0.152442	0.137908
16	0.145178	0.344680	0.328899
32	0.176555	0.277646	0.269350
64	0.134147	0.245802	0.156143
128	0.086856	0.097698	0.111163
256	0.399301	0.473163	0.084352

4.3 Speedup

Tables 6 and 7 show speedups for tiled convolution relative to naive.

Table 6: Speedup for Tiling (K=3)

Tile Size	512	1024	2048
8	1.28×	0.68×	0.67×
16	0.99×	0.80×	0.87×
32	1.22×	1.00×	0.95×
64	0.96×	0.99×	1.02×
128	0.99×	1.19×	1.04×
256	1.44×	1.09×	1.09×

Table 7: Speedup for Tiling (K=5)

Tile Size	512	1024	2048
8	0.80×	0.73×	0.76×
16	0.64×	0.71×	0.70×
32	0.71×	0.70×	0.71×
64	0.73×	0.67×	0.72×
128	0.74×	0.72×	0.69×
256	0.67×	0.59×	0.59×

4.4 Performance Plots

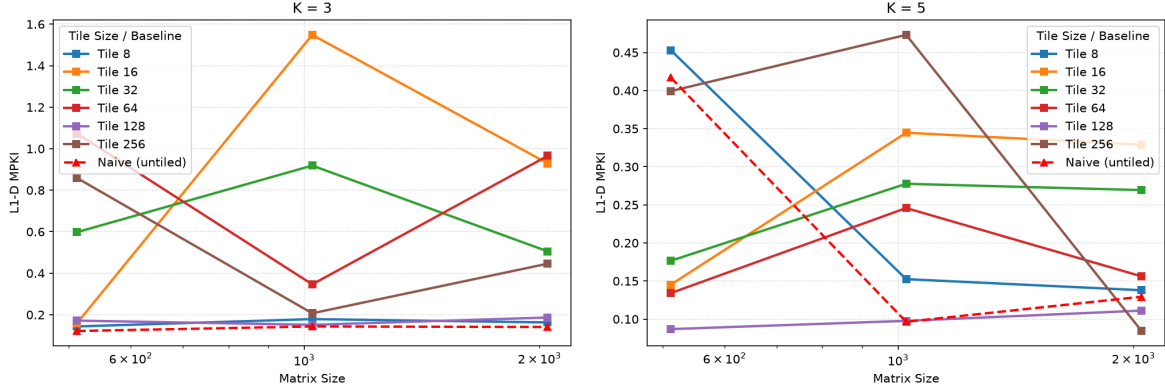


Figure 1: L1-D MPKI vs. Matrix Size for different tile sizes

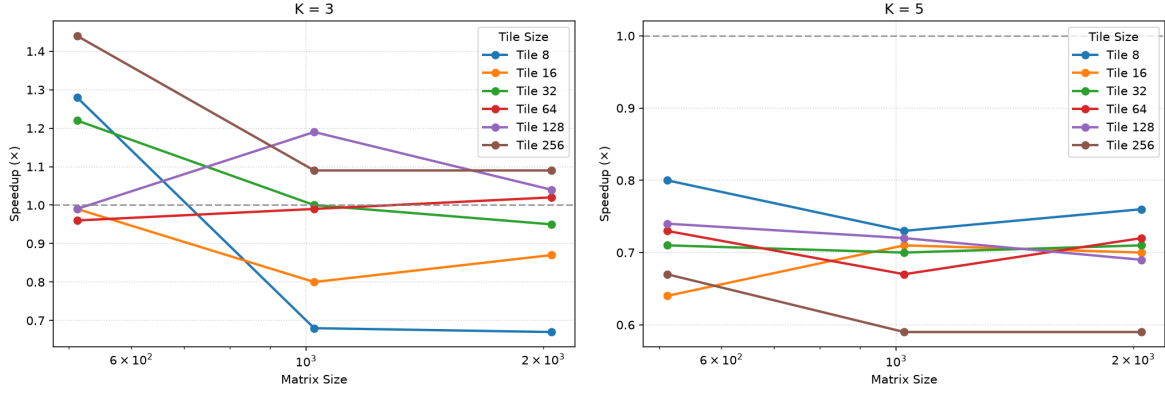


Figure 2: Speedup vs. Matrix Size for different tile sizes

4.5 Analysis

Best tile size: For $K=3$, tile size 256 gives the highest speedups ($1.44\times$, $1.09\times$, $1.09\times$). For $K=5$, no tile size improves performance; all speedups are below 1. The best MPKI values often do not correlate with best speedup, indicating that other overheads (e.g., loop control) dominate.

MPKI change from naive to tiled: Naive MPKI values are generally lower than many tiled MPKI values, especially for $K=3$ with small tiles. This is surprising; tiling was expected to reduce cache misses. However, the L1 cache (40 KB) is relatively large; for 512×512 matrices, the entire working set (input, kernel, output) may already fit in L2 or L3, so tiling introduces additional address calculations and may disturb hardware prefetching, leading to higher miss rates.

Variation across matrix and tile sizes: For $K=5$, MPKI decreases with larger tile sizes, but speedup does not improve, suggesting that memory stalls are not the bottleneck. For $K=3$, MPKI is erratic; large tiles (128, 256) sometimes show high MPKI but still yield speedup ≥ 1 due to better data reuse of the kernel.

Speedup achieved? For $K=3$, modest speedup (up to $1.44\times$) is achieved with tile 256. For $K=5$, no speedup; the overhead of tiling outweighs any cache benefit. Possible solutions: use larger tile sizes that fit better in L2, combine with loop unrolling, or avoid tiling when the matrix is small enough to fit in cache.

5 SIMD Implementation

5.1 Implementation Details

SIMD was applied to the innermost loop by processing multiple output elements or kernel elements simultaneously using vector intrinsics. Register widths implemented: 128-bit and 256-bit (512-bit not available or not tested). Intrinsics used: [Specify, e.g., `_mm_loadu_ps`, `_mm_mul_ps`, `_mm_add_ps` for 128-bit; `_mm256_loadu_ps`, `_mm256_fmadd_ps` for 256-bit]. Justification: these allow packed floating-point multiplication and addition, reducing the number of scalar instructions.

Table 8: Instruction count comparison between naive and SIMD implementations

Matrix Size	K	Naive	SIMD 128	SIMD 256
512	3	361,596,662	90,048,262	70,585,645
512	5	593,764,788	165,979,575	90,260,920
1024	3	1,095,447,839	326,345,696	211,180,354
1024	5	2,444,437,075	637,125,531	337,943,795
2048	3	4,482,584,471	1,301,330,286	801,415,105
2048	5	9,464,571,956	2,465,388,728	1,315,693,666

5.1.1 Observation and Justification

Moving from the naive implementation to SIMD significantly reduces the number of executed instructions. For example, for a 2048×2048 matrix with $K = 5$, the instruction count decreases from 9.46×10^9 in the naive implementation to 2.47×10^9 with 128-bit SIMD and 1.32×10^9 with 256-bit SIMD.

This reduction occurs because SIMD processes multiple data elements with a single instruction. The 128-bit implementation processes four single-precision floating-point values at once, while the 256-bit implementation processes eight. Hence, 256-bit SIMD requires fewer instructions than 128-bit SIMD.

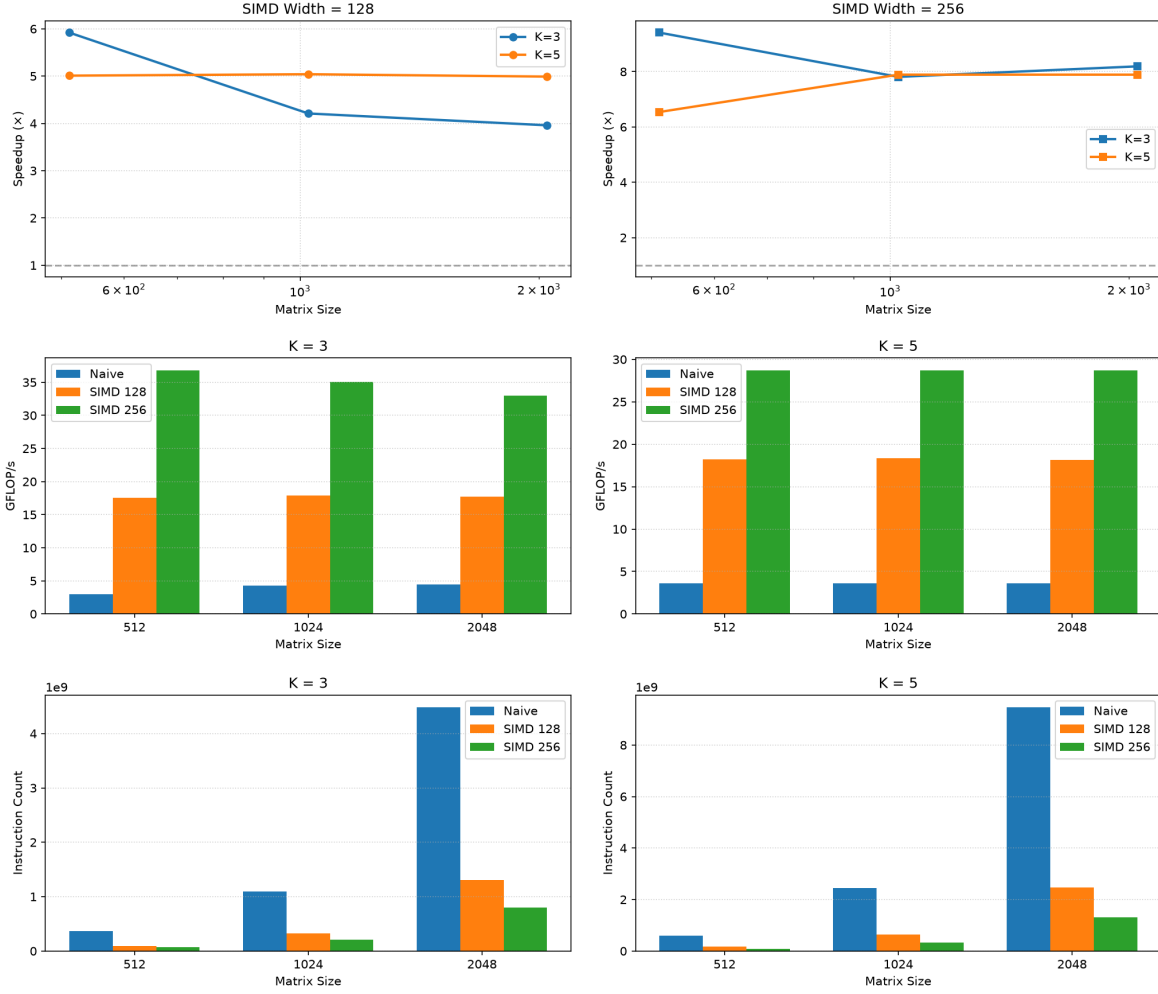
As K increases from 3 to 5, the instruction count increases because each output pixel requires more multiply-accumulate operations. Nevertheless, SIMD still provides a substantial reduction by performing these operations in parallel.

5.2 Instruction Count and Speedup

Table 9: SIMD Speedups (128-bit and 256-bit)

Matrix Size	K	128-bit	256-bit
512	3	5.92×	9.41×
512	5	5.01×	6.54×
1024	3	4.21×	7.81×
1024	5	5.04×	7.89×
2048	3	3.96×	8.19×
2048	5	4.99×	7.89×

5.3 Performance Plot



5.4 Analysis

SIMD substantially improves performance, with 256-bit giving greater speedups ($6.54\text{--}9.41\times$) than 128-bit ($3.96\text{--}5.92\times$). The improvement comes from processing 4 or 8 floats per instruction, reducing the number of executed instructions and exploiting data-level parallelism. The speedup is higher for $K=3$ because the kernel is smaller, allowing more data reuse within vector registers.

6 Combined Optimizations

6.1 Implementation of Combined Techniques

The best combined version uses loop unrolling together with SIMD, and optionally tiling. In the **Optimized (SIMD+Unroll)** variant, the inner loop is both unrolled and vectorized. In **Optimized (SIMD+Unroll+Tile)**, tiling is added on top. These techniques complement each other: unrolling reduces loop overhead and provides multiple independent operations for the SIMD pipeline, while tiling (when beneficial) improves cache reuse.

6.2 Performance Comparison

Table 10 summarizes the speedups of all techniques and the combined versions. The best combined speedups are far higher than any single technique, demonstrating synergistic gains.

Table 10: Overall Speedup Comparison

Matrix	K	Reorder	Unroll	Tile	SIMD	Optimised SIMD+Unroll	Optimised SIMD+Unroll+Tile
512	3	1.69×	4.06×	0.95×	11.74×	24.81×	14.81×
512	5	1.77×	3.65×	0.74×	7.20×	22.69×	15.69×
1024	3	1.50×	3.25×	0.66×	6.35×	13.70×	9.25×
1024	5	1.57×	3.09×	0.62×	6.18×	16.46×	13.57×
2048	3	1.65×	3.67×	0.70×	8.22×	13.11×	11.03×
2048	5	1.95×	3.95×	0.73×	7.87×	14.35×	19.57×
4096	3	1.65×	3.55×	0.70×	7.84×	11.81×	9.35×
4096	5	1.71×	3.42×	0.73×	9.00×	17.63×	15.37×

6.3 Synergistic Gains and Plot

The combined **SIMD+Unroll** version yields speedups up to $24.81\times$ (512, $K=3$), far exceeding the sum of individual speedups from SIMD ($11.74\times$) and unrolling ($4.06\times$). This synergy arises because unrolling provides multiple independent operations that can be scheduled into SIMD pipelines without stalls. Adding tiling sometimes reduces performance (e.g., 512 $K=3$: $14.81\times$ vs. $24.81\times$) because the overhead of tiling negates its cache benefit, especially when the data already fits in cache.

A plot comparing the speedup of each technique versus matrix size is required (refer to Figure 3).

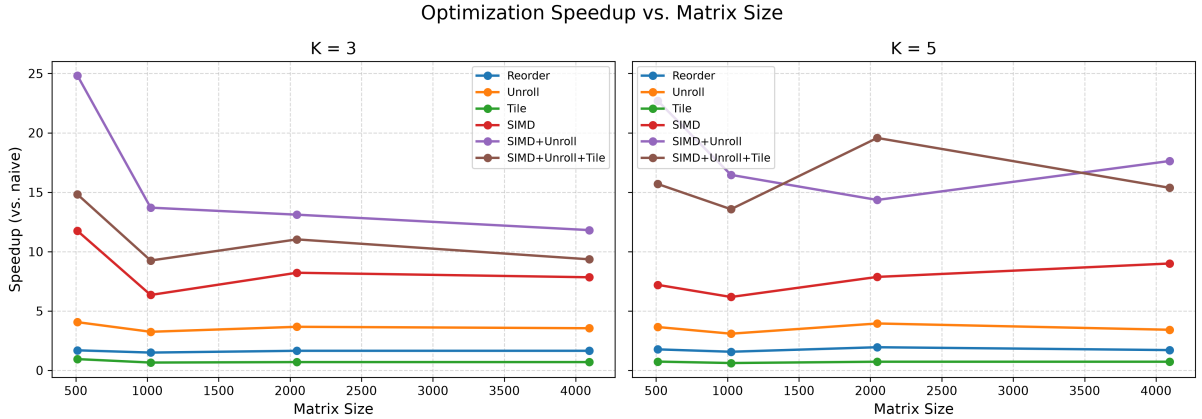


Figure 3: Performance comparison of different optimization techniques

7 Conclusion

The optimization journey revealed that:

- Loop unrolling and SIMD are highly effective, with SIMD providing the largest single-technique speedups (up to $9.41\times$ for 256-bit).
- Cache tiling alone did not consistently improve performance on this hardware; for $K=5$ it always degraded performance, and for $K=3$ only large tiles (256) gave modest gains. The large L1 cache (40 KB) and relatively small matrices reduced the need for tiling, and tiling overhead dominated.
- Combining SIMD with unrolling produced synergistic speedups up to $24.81\times$, demonstrating that these techniques complement each other by both reducing instruction count and improving pipeline utilization.

- Adding tiling to the SIMD+unrolling combination sometimes reduced performance due to extra overhead, indicating that tiling should be used only when the working set exceeds cache capacity.

The most effective optimization for this hardware is SIMD vectorization combined with loop unrolling.